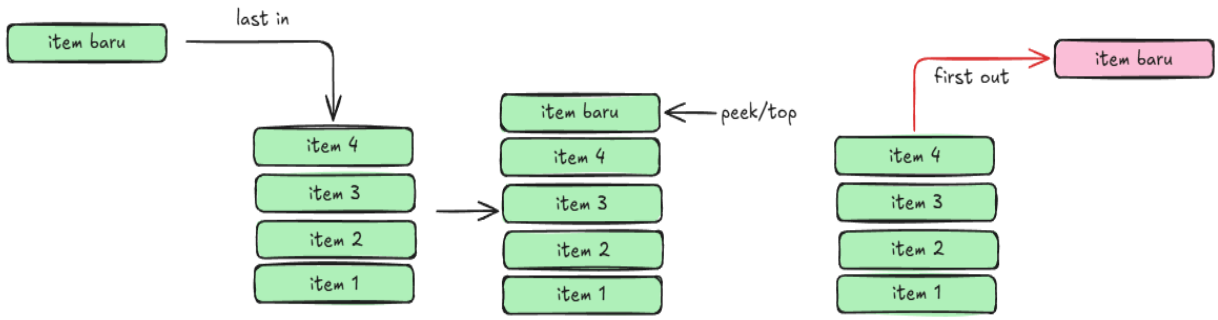
Stack dan Queue

No 4

Struktur Data

Stack

Stack (tumpukan) adalah struktur data linier yang menggunakan prinsip LIFO (Last In First Out). Sederhananya, stack dapat diibaratkan sebagai tumpukan item yang disusun ke atas.



Seperti contoh di gambar, item pertama yang ditumpuk akan berada di bagian paling bawah, sementara item-item berikutnya diletakkan di atasnya. Saat mengambil item, yang pertama diambil adalah item yang terakhir ditumpuk. Berikut adalah beberapa operasi yang dapat dilakukan pada stack :

1. Push

   Menambah sebuah item pada stack, jika penambahan dilakukan pada kondisi stack penuh maka kondisi ini disebut overflow.

2. Pop

   Menghapus sebuah item pada stack , hal ini dilakukan pada tumpukan yang paling atas, jika penghapusan dilakukan pada saat kondisi stack kosong maka kondisi ini disebut underflow.

3. Peek or Top

   Melihat item teratas pada stack.

Kode Program Stack Dengan Array.

Persiapan Program (Setup)

```
#include <iostream>
#define MAX 100
using namespace std;
int stack[MAX];
int top = -1; // Menandakan elemen teratas pada stack
```

Dalam kode ini, telah disediakan beberapa fungsi dasar pada stack. Selain itu, program juga menangani kondisi stack overflow (jika elemen yang dimasukkan melebihi kapasitas maksimum) dan stack underflow (jika dilakukan penghapusan pada stack kosong). Dibawah ini merupakan kode untuk program stack dengan array:

- **push()** untuk menambahkan elemen ke dalam stack.

```
void push(int value) {
   if (top >= MAX - 1) {
       cout << "Stack overflow" << endl;
   }
   else {
       top++;
       stack[top] = value;
       cout << value << " pushed into stack" << endl;
   }
}
```

- **pop()** untuk menghapus dan mengembalikan elemen teratas dari stack.

```
int pop() {
   if (top < 0) {
       cout << "Stack underflow" << endl;
       return -1;
   }
   else {
       int value = stack[top];
       top--;
       return value;
   }
}
```

- **peek()** untuk melihat elemen teratas tanpa menghapusnya.

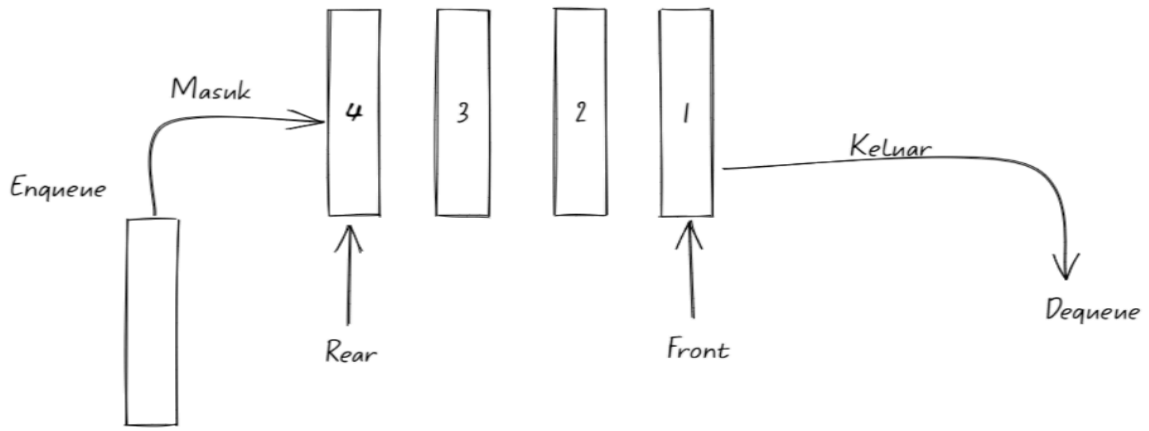
```
int peek() {
   if (top < 0) {
       cout << "Stack is empty" << endl;
       return -1;
   }
   else {
       return stack[top];
   }
}
```

- **isEmpty()** untuk memeriksa apakah stack kosong atau tidak.

```
bool isEmpty() {
   return (top < 0);
}
```

QUEUE

Queue atau antrian dalam struktur data adalah kumpulan elemen di mana penambahan elemen hanya dapat dilakukan pada salah satu ujung yang disebut sisi belakang (rear), sementara penghapusan elemen dilakukan melalui ujung lain yang disebut sisi depan (front). Prinsip dasar dari queue adalah FIFO (First In First Out), yang berarti elemen yang pertama kali dimasukkan akan menjadi yang pertama diakses atau dikeluarkan.



Sebagai analogi, hal ini serupa dengan antrian di loket pembelian tiket kereta, di mana orang yang datang lebih dulu akan dilayani lebih dahulu, dan akan selesai lebih cepat daripada orang yang datang belakangan. Berikut adalah beberapa operasi yang dapat dilakukan pada queue:

1. Enqueue

   Penambahan item pada queue, penambahan dilakukan pada ekor antrian jika penambahan dilakukan pada saat queue penuh maka kondisi ini disebut overflow.

2. Dequeue

   Penghapusan item pada queue, penghapusan dilakukan pada kepala (Front) antrian, jika penghapusan dilakukan pada saat queue kosong maka kondisi ini disebut underflow.

3. Front Mengambil item antrian pertama pada queue.
4. Rear Mengambil item antrian terakhir pada queue.

Kode Program Queue Dengan Array

Persiapan Program (Setup)

```
#include <iostream>
#define MAX 100
using namespace std;
int queue[MAX];

int front = -1, rear = -1;
bool isFull() {
   // Jika posisi rear sudah di indeks terakhir (MAX-1), maka penuh
   return rear == MAX - 1;
}
bool isEmpty() {
   // Kosong jika front masih -1 ATAU front sudah melewati
   // posisi rear (setelah banyak dequeue)
   return front == -1 || front > rear;
}
```

Dalam program ini, disediakan beberapa operasi dasar pada queue. Program ini juga menangani kondisi queue overflow (apabila antrian penuh) dan queue underflow (apabila antrian kosong). Selain itu, terdapat mekanisme reset otomatis ketika queue kembali kosong setelah dilakukan operasi dequeue. Dibawah ini merupakan kode untuk program stack dengan array:

- **enqueue()** untuk menambahkan elemen ke dalam antrian,

```
void enqueue(int value) {
   // jika queue full hentikan program
   if (isFull()) {
       cout << "Queue overflow" << endl;
       return;
   }

   // Jika elemen pertama masuk, set front
   // ke indeks 0 agar queue aktif
   if (front == -1) {
       front = 0;
   }
   rear++; // Geser posisi rear ke kanan untuk tempat elemen baru
   queue[rear] = value; // Masukkan nilai ke posisi rear yang baru
   cout << value << " enqueued into queue" << endl;
}
```

- **dequeue()** untuk menghapus elemen dari bagian depan antrian,

```
int dequeue() {
   // jika queue kosong hentikan program
   if (isEmpty()) {
       cout << "Queue underflow" << endl;
       return -1;
   }

   // Ambil nilai yang ada di posisi paling depan (front)
   int value = queue[front];

   // Geser front ke kanan (elemen dianggap "keluar" dari antrean)
   front++;
   // Cek apakah setelah dihapus antrean jadi kosong total
   if (isEmpty()) {
       // Jika ya, reset kedua penanda ke posisi awal (-1)
       front = rear = -1;
   }
   return value;
}
```

- peek() untuk melihat elemen terdepan tanpa menghapusnya,

```
int peek() {
    if (isEmpty()) {
        cout << "Queue is empty" << endl;
        return -1;
    }
    // Mengembalikan nilai di posisi front saat ini
    return queue[front];
}
```

- **display()** untuk menampilkan seluruh elemen queue.

```
void display() {
    if (isEmpty()) {
        cout << "Queue is empty" << endl;
        return;
    }
    cout << "Queue elements: ";
    // Loop mulai dari posisi front sampai ke rear
    for (int i = front; i <= rear; i++) {
        cout << queue[i] << " ";
    }
    cout << endl;
}
```